ReSource PK

Clone se complete local run aur Week 1 Week 2 verification tak

Beginner guide • Windows aur Git Bash ke liye • 11 October 2026

Aap ne project pehle kabhi run nahin kiya? Is guide ko sequence mein follow karein. Pehle tools install honge, phir dono repositories clone hongi, database aur environment set hoga, phir seller, reviewer aur buyer ka poora flow check hoga.

Explanations easy Hinglish mein hain. Commands aur screen ke labels English mein rakhe hain taake aap unhein seedha copy ya screen par identify kar sakein.

Sabse pehle yeh samjhein

Product add ka kaam implemented hai: Owner ya Manager draft banata hai, images aur quantity price tiers add karta hai, platform reviewer publish karta hai, aur buyer published listing discover karta hai. Aapko product form khud code nahin karna; apna test product form mein enter karna hai.

Week 1 aur Week 2 ke features kaafi had tak implemented aur automated checks se verified hain. Hosted email/storage/scheduler evidence, staging auth sign off aur kuch repository settings abhi pending hain. Isliye in weeks ko fully released ya har environment mein error free nahin keh sakte.

Guide snapshot	Exact source
Backend	feature/week-2-acceptance e6a26171f521a75f0c94b07b6c066191dd27aa62
Frontend	feature/week-2-acceptance f4df243ef18c5ae5eec327f73264d3b63574ec19
Review state	Backend PR 8 aur Frontend PR 6 abhi open hain. Latest work main par merged nahin hai.

Is guide mein koi asli password, API key ya production token nahin diya gaya. Har placeholder ko aap apne computer par fill karein. Secrets chat, screenshots aur GitHub mein share na karein.

1 Guide ko kaise follow karna hai

Phase	Sections aur outcome
Samajhna	2–4: implemented work, remaining work, roles aur architecture.
Pehla run	5–11: tools, clone, PostgreSQL, environment, migrations, demo seed, two servers.
Catalog check	12–17: login, seller product, moderation, discovery, requirements, imports.
Real services	18–21: Gmail, real signup, Manager/Staff invitations, Supabase images.
Deep checks	22–26: SQL checks, auth/security, errors, responsive/browser checks, automated suites.
Hosted aur handover	27–30: hosted sign off, troubleshooting, restart, final checklist aur source links. Section 31 optional Supabase DB path.

Teen passes mein kaam karein

Pass A: Sections 5–17. Local demo se UI aur catalog ka working flow dekhein. Is pass mein email aur image storage credentials zaroori nahin. Images ka successful upload is pass se prove nahin hota.

Pass B: Sections 18–21. Apne controlled inboxes aur private test bucket ke saath real verification, reset, invitations aur images check karein.

Pass C: Sections 22–30. Database queries, security boundaries, automated checks aur hosted acceptance evidence collect karein. Jo provider available nahin, use BLOCKED likhein; PASS nahin.

Result likhne ka simple rule

Har checkpoint ke liye PASS, FAIL, BLOCKED ya NOT RUN likhein. PASS tab jab expected result actual screen/API/database mein mila. FAIL par exact step, time, role, URL, error code aur request ID note karein. Credentials ya email links ke tokens evidence mein redact karein.

Commands sirf Git Bash mein chalayein. Ek block ki har line Enter se chal sakti hai. Terminal prompt ka $ copy nahin karna; blocks mein prompt diya hi nahin hai. Code blocks SQL ya .env hon to unhein terminal mein run na karein; unka destination paas likha hai.

2 Week 1 mein kya implement ho chuka hai

Area	Current implementation aur aapka check
Identity	Buyer/Seller signup, email verification, resend, login, logout aur forgotten password reset. Real inbox test ke liye SMTP chahiye.
Sessions	Short access JWT, server owned refresh sessions, rotation/revocation, refresh cookie aur reload par session restore.
Organizations	Seller organization, Owner membership, Manager/Staff invitations, email bound acceptance aur organization isolation.
Permissions	Active verified account aur current membership recheck; Owner vs Manager vs Staff vs platform reviewer boundaries.
Database	Users, sessions, email tokens, organizations, memberships, invitations, platform assignments, delivery records aur audit events.
API foundation	Versioned API, schema validation, stable errors, request IDs, /health aur database backed /ready.
Frontend	Public landing, authentication screens, buyer/organization/platform shells, workspace switcher aur responsive navigation.
Engineering	README linked baseline documents, lint/format, CI checks, dependency/secret checks aur reviewed branches.

Owner/CEO ka alag login application nahin hai. Seller signup karne wala owner organization workspace mein aata hai. Manager aur Staff ko wahi workspace unki permission ke mutabiq milta hai. Platform reviewer organization Manager se alag role hai.

Week 1 ke pending gates

Branch protections/review governance, final managed provider decisions aur hosted frontend/API/database authentication journey ka sign off partial hai. Full membership administration, ownership transfer, suspension UI aur complete financial dashboards delivered nahin hain.

Demo accounts verified state mein seed hote hain. Demo login ko real email verification ya reset delivery ka proof na samjhein.

3 Week 2 mein kya implement ho chuka hai

Area	Current implementation
Seller catalog	Create/edit drafts, quantity/MOQ, unit, exact decimal price, material/condition aur optional color/composition/location.
Price tiers	Up to 10 quantity thresholds, increasing quantities aur prices jo increase nahin hote; exact paisa storage.
Moderation	Draft → pending review → published/rejected; rejected draft correction; terminal archive; version conflict protection.
Buyer discovery	Public search/filter/sort/pagination, published listing detail page aur direct reload.
Requirements	Create, list/paginate, explainable material/unit/quantity/tier price matching, pause/resume aur close.
Imports	CSV/XLSX template, preview, row errors, explicit confirmation, draft creation aur replay protection.
Images	Validated raster input, private WebP storage, alt text, signed reads, delete aur durable recovery cleanup jobs. Provider credentials required.
Maintenance	CLI cleanup aur authenticated internal cleanup endpoint; daily Vercel cron definition. Real schedule execution pending.
Demo and checks	Opt in 4 synthetic suppliers, 20 listings, 6 accounts; PostgreSQL integration aur Chromium journeys.

Abhi kya baaki hai

Week 2 T 102 partial hai: live upload → authorized signed read → remove/cleanup, hosted seller → reviewer → buyer flow aur actual scheduled cleanup logs chahiye. Vercel team access last check par 403 tha; workspace mein configured SMTP/storage credentials nahin the.

Week 3: group buying pools, stock reservations, checkout, payments, callbacks, ledger aur refunds. Week 4: fulfilment, disputes, verified ratings, material passports/blockchain, real impact metrics aur reports/CSV/PDF exports. Week 5: full cross browser/device/load, backup restore, observability aur production release drills.

Orders, My purchases, Impact, Analytics aur kuch admin navigation abhi shell/placeholders hain. Empty values ko error ya implemented payments na samjhein. Review queue aur catalog actual features hain.

4 Architecture roles aur project rules

Layer	Stack aur data boundary
Frontend	React 18 + Vite 8, JavaScript, responsive web UI. API base VITE_API_URL se aata hai.
Backend	Node 24 recommended, Express 5 modular monolith; route/controller → service → repository/query.
Database	PostgreSQL, private app schema, numbered migrations, constraints, transactions, parameterized SQL.
Auth	Application owned users/sessions, bcrypt password hashes, JWT and rotating refresh cookie. Supabase Auth signup nahin.
Email and images	Nodemailer Gmail SMTP; Supabase Storage private bucket. Keys sirf backend.

Role	Catalog	Team invitations	Entry
Buyer	Discover + own requirements	None	Buyer signup/demo buyer
Owner or CEO	Own org create/edit/submit/archive	Manager + Staff	Seller signup/demo owner
Manager	Own org create/edit/submit/archive	Staff only if granted	Owner email invitation
Staff	Organization shell; seller catalog denied currently	None	Owner/delegated Manager invite
Platform reviewer	Pending queue + publish/reject	No org grant implied	Trusted platform assignment/demo admin

README ke docs/README.md, prd.md, srs.md, architecture.md, design.md, rules.md, tasks.md, memory.md aur analytics.md dono repos mein baseline hain. Feature ko current code aur acceptance criteria dono se compare karein. Applied migrations edit na karein; new schema change new numbered migration se hota hai.

Runtime stock, account roles, money/impact numbers aur provider URLs fabricated fallback data se nahin aate. Demo fixtures explicit seed mein hain. UI labels, design tokens aur reviewed validation limits constants ho sakte hain; woh secret ya fabricated business record nahin hain.

5 Windows par tools install karein

1. Internet aur GitHub access ready karein. Browser mein dono repository URLs open karein. Private repository ho to authorized GitHub account se sign in karein.

2. Git for Windows official installer install karein. Default options theek hain; Git Bash available honi chahiye. GitHub authentication browser se karein, password/PAT ko clone URL mein na daalein.

3. Node.js download page par Node 24 LTS aur Windows installer choose karein. npm included hai. Install ke baad terminals close aur reopen karein.

4. Visual Studio Code install karein. Terminal menu → New Terminal. Terminal dropdown → Select Default Profile → Git Bash; phir naya terminal kholein.

5. PostgreSQL 17 ka Windows installer official PostgreSQL link se install karein. PostgreSQL Server aur pgAdmin include karein. Port 5432 rakhein; postgres administrator password privately save karein. Stack Builder extras is guide ke liye zaroori nahin.

6. Chrome ya Edge use karein. Automated tests baad mein apna Chromium install karenge.

Git Bash mein tool versions check karein

```
git --version
node --version
npm --version
```

Check: Git aur npm version print hon; Node v24.x recommended. Backend >=24 aur <27 allow karta hai. Node 18/20/22 par yeh snapshot mat run karein.

Git installer

Node installer

PostgreSQL Windows installer

VS Code installer

Mac/Linux par bhi same app commands chalti hain, lekin Windows installation aur /c/Projects path applicable nahin. Apne OS ke official installers aur ~/Projects folder use karein; PostgreSQL local aur same database names rakhein.

6 Dono repositories ki correct branch clone karein

Destination: Git Bash. Yeh fresh folder banata hai. Agar same folder mein pehle clone hai to delete na karein; naya preview folder choose karein. Commands mein backslash line continuation hai; poora block paste karein.

```
mkdir -p /c/Projects/ReSourcePK-preview
cd /c/Projects/ReSourcePK-preview
git clone --branch feature/week-2-acceptance --single-branch \
  https://github.com/Raazia-Imran/ReSourcePK_Backend.git backend
git clone --branch feature/week-2-acceptance --single-branch \
  https://github.com/Raazia-Imran/ReSourcePK_Frontend.git frontend
```

Backend aur frontend ko verify karein

```
cd /c/Projects/ReSourcePK-preview/backend
git branch --show-current
git rev-parse HEAD
git status --short
cd ../frontend
git branch --show-current
git rev-parse HEAD
git status --short
```

Check: Dono branch names feature/week-2-acceptance. Fresh checkout ka status empty. Guide cover ke commits exact snapshot hain; branch aage badhe to HEAD differ ho sakta hai. Sirf main clone karne se latest Week 2 work miss ho sakta hai.

Exact reproducibility chahiye to respective repo mein git switch --detach ke baad cover ka full commit paste karein. Is detached checkout ko sirf run/check ke liye use karein; coding ke liye new branch banayein. Normal beginner path feature branch hai.

1. VS Code → File → Open Folder → C:\Projects\ReSourcePK-preview choose karein. Dono sibling folders Explorer mein dikhne chahiye.

2. backend/README.md aur frontend/README.md se docs/README.md open karein. Old patch instructions dobara apply na karein; code already consolidated hai.

3. VS Code mein do Git Bash terminals rakhein: Backend aur Frontend. Agle sections batate hain kis terminal/folder mein command chalani hai.

Clone error Repository not found ka matlab URL/private access check. Branch not found par PR links section 30 dekhein; branch merge/delete ho gayi ho to merged revision verify karke clone karein. Bina verify kiye unrelated branch select na karein.

7 Local PostgreSQL aur preview database banayein

Recommended pehla run disposable local database mein hai. Is se staging/production users aur inventory par aapke experiments ka asar nahin hoga. PostgreSQL password website login password se alag hai.

1. Windows Start menu → pgAdmin 4 open karein. Agar master password poochhe to pgAdmin saved connection protection ke liye apna password choose karein; yeh postgres password se alag ho sakta hai.

2. Left panel Servers expand karein. Installed PostgreSQL server connect karein, postgres password enter karein. Server missing ho: Servers right click → Register → Server; Name Local PostgreSQL; Connection Host 127.0.0.1, Port 5432, Maintenance database postgres, Username postgres.

3. Databases right click → Create → Database. Name resourcepk_preview, Owner postgres, Save. Isi database par right click → Query Tool kholein.

4. Neeche ka SQL Query Tool mein paste karke Execute/F5 karein. Yeh local Supabase compatible NOLOGIN roles banata hai; in roles ko password ya admin privileges mat dein.

```
DO $$ BEGIN
  CREATE ROLE anon NOLOGIN;
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
DO $$ BEGIN
  CREATE ROLE authenticated NOLOGIN;
EXCEPTION WHEN duplicate_object THEN NULL; END $$;
```

Check: SQL successfully completes. Database resourcepk_preview hai; yahan abhi app tables nahin honge. Migration next section ke baad banayegi.

Password ko connection URL mein safely likhein

Special characters @, #, %, /, : aur spaces URL mein encode hone chahiye. VS Code terminal mein neeche command run karein, apna password sirf hidden prompt mein type karein. Result ko locally .env mein use karein; share na karein. Input echo nahin hoga.

```
read -s -p "Postgres password: " QA_DB_PASS
echo
export QA_DB_PASS
node -e "console.log(encodeURIComponent(process.env.QA_DB_PASS))"
unset QA_DB_PASS
```

Database URL local preview: postgresql://postgres:YOUR_ENCODED_DB_PASSWORD@127.0.0.1:5432/resourcepk_preview. Replace full placeholder; localhost par sslmode=require add na karein. Installed port different ho to 5432 replace karein.

8 Backend dependencies aur environment set karein

Destination: Backend Git Bash terminal. Ignore scripts project security rule ka hissa hai; npm install ya unrestricted lifecycle hooks se replace na karein.

```
cd /c/Projects/ReSourcePK-preview/backend
npm ci --ignore-scripts
cp .env.example .env
```

VS Code Explorer mein backend/.env open karein. Neeche initial local configuration fill karein. YOUR_ENCODED_DB_PASSWORD section 7 ka value hai; YOUR_RANDOM_JWT_SECRET generate karke replace hoga. Quotes ke andar secrets rakh sakte hain; example placeholders as is mat chhodein. PDF mein URL visually wrap ho sakta hai, actual .env file mein DATABASE_URL ek single line par rakhein.

```
NODE_ENV=development
AUTH_REQUESTS_PER_WINDOW=12
PORT=5000
DATABASE_URL="postgresql://postgres:YOUR_ENCODED_DB_PASSWORD@127.0.0.1:5432/
resourcepk_preview"
JWT_ACCESS_SECRET=YOUR_RANDOM_JWT_SECRET
ACCESS_TOKEN_TTL=15m
REFRESH_TOKEN_DAYS=30
APP_URL=http://localhost:3000
CORS_ORIGINS=http://localhost:3000
EMAIL_FROM_NAME=ReSource PK
TRUST_PROXY_HOPS=0
DATABASE_POOL_MAX=5
PUBLIC_READS_PER_MINUTE=300
```

Random JWT secret generate karein

```
node -e "console.log(require('node:crypto').randomBytes(48).toString('hex'))"
```

Printed value copy karke JWT_ACCESS_SECRET replace karein, save Ctrl+S. Secret minimum 32 characters chahiye. Gmail aur Supabase settings abhi commented rehne dein; partial set of values config error de sakta hai. CRON_SECRET baad mein independent random value hoga.

Check: npm ci successful exit; .env backend root mein, .env.txt nahin. git status --short mein .env appear nahin hona chahiye. File ignore ho rahi hai, iska matlab secrets safe share karna nahin.

DATABASE_URL mein actual database admin password hai. JWT secret website password nahin. GMAIL_APP_PASSWORD normal Google password nahin. SUPABASE_SERVICE_ROLE_KEY backend privileged provider key hai. In sab ko frontend VITE variables mein kabhi na daalein.

9 Migrations aur demo accounts prepare karein

Destination: Backend terminal, normal local .env. NODE_ENV production mat set karein. Seed sirf resourcepk_preview disposable database par chalayein.

```
npm run db:migrate
npm run db:migrate
```

Check: First run 001–005 Applied messages de sakta hai. Second run already applied migrations skip karega; exit success aur koi duplicate tables nahin. app.schema_migrations mein five records verify section 22 se karein.

Apna demo password choose karein

Password minimum 10 aur maximum 72 UTF 8 bytes; uppercase, lowercase aur number required. Is ko privately save karein. Neeche prompt hidden hai; koi built in/default password nahin.

```
export ALLOW_DEMO_SEED=true
read -s -p "Choose strong demo password: " DEMO_PASSWORD
echo
export DEMO_PASSWORD
npm run db:seed:demo
npm run db:seed:demo
node scripts/verify-demo.js
unset ALLOW_DEMO_SEED
unset DEMO_PASSWORD
```

Check: Verification says 4 synthetic suppliers, 20 labelled listings, 6 demo accounts. Second seed duplicate records nahin banata. Demo stock badge visible hoga. Seeded listings already published hain aur unmein images nahin hain.

Demo account	Use
demo-buyer@resourcepk.example.invalid	Buyer discovery/requirements
demo-admin@resourcepk.example.invalid	Local platform reviewer
demo-owner-1@resourcepk.example.invalid	Owner of demo-organization-1
demo-owner-2@resourcepk.example.invalid	Separate org isolation check
demo-owner-3@resourcepk.example.invalid	Third supplier
demo-owner-4@resourcepk.example.invalid	Fourth supplier

Sab six accounts ka initial password aapka chosen DEMO_PASSWORD hai. Emails deliberately non deliverable hain. Manager/Staff seeded nahin hain. Seed replay password rotate nahin karta: variable change se existing account ka password change nahin hota. Forgot password test real inbox account par karein.

10 Frontend set karein aur two servers start karein

Destination: Frontend terminal. .env frontend root mein create karein, phir VS Code mein uski only required value verify karein.

```
cd /c/Projects/ReSourcePK-preview/frontend
npm ci --ignore-scripts
cp .env.example .env
```

Destination: frontend/.env file. Is mein secrets ya DATABASE_URL nahin honi chahiye.

```
VITE_API_URL=http://localhost:5000/api/v1
```

Backend terminal mein server start

```
cd /c/Projects/ReSourcePK-preview/backend
npm run dev
```

Frontend terminal mein server start

```
cd /c/Projects/ReSourcePK-preview/frontend
npm start
```

1. Dono terminal processes running rehne dein. In terminals mein aur command tab tak na type karein jab tak Ctrl+C se process stop na karein.

2. Browser mein http://localhost:5000/health open karein. Expected JSON status ok.

3. http://localhost:5000/ready open karein. Expected status ready; yeh actual database connection check hai.

4. http://localhost:3000 open karein. Public home visible hona chahiye. Marketplace par synthetic listings dikhni chahiye.

5. Vite agar 3001 par shift ho gaya to previous 3000 process stop karein aur 3000 dobara use karein. Is guide mein frontend URL, CORS aur APP_URL sab exactly localhost:3000 hain.

Check: UI loads, API ready hai, marketplace data database se aa raha hai. Health ok lekin ready fail ho to database still broken hai; UI check se pehle section 28 troubleshoot karein.

Environment file change ke baad backend Ctrl+C → npm run dev aur frontend Ctrl+C → npm start se restart karein. Production build mein VITE_API_URL build time par embed hota hai, isliye hosted change par rebuild required hai.

11 First smoke check aur log collection

Check	Action aur expected result
Public home	Open /. Links aur auth buttons usable; no fabricated live financial/impact figures.
Marketplace	Open /marketplace. Demo records labelled; no login needed for published listings.
Detail reload	View listing click; URL /marketplace/listings/<UUID>. Browser reload still detail dikhaye.
API ready	/health ok, /ready ready. Marketplace request GET /api/v1/listings is 200.
Sign in page	/login opens email/password form. Sign out state mein /workspace login par redirect kare.
Browser errors	F12 → Console aur Network. New uncaught errors, failed JS files ya repeat failing API requests investigate karein.

Browser DevTools ko beginner tareeqe se use karein

1. F12 press karein, Network tab open karein. Clear button click karein, page reload karein. Filter Fetch/XHR use karein.

2. Ek feature click karein. Relevant request select karke Headers mein URL/method/status aur Response mein error code dekhein.

3. Backend errors mein x-request-id header ya error.correlationId record karein. Backend terminal mein same ID search karne se request trace hota hai.

4. Evidence share karne se pehle Authorization, Cookie, Set Cookie, tokens, signed URLs aur passwords remove karein. HAR exports normally secrets contain kar sakte hain; raw HAR share na karein.

Signed out initial session refresh ka 401 expected ho sakta hai. Login ke baad repeated 401, infinite refresh ya wrong workspace expected nahin. Missing storage configuration ka upload failure expected in Pass A, lekin upload feature PASS nahin count hoga.

Check: Smoke result: Date _____ Backend HEAD _____ Frontend HEAD _____ /ready _____ Home _____ Marketplace _____ Detail reload _____ Console _____

12 Seller Buyer aur reviewer login karein

1. Open http://localhost:3000/login. Demo Owner 1 ka email aur section 9 mein chosen demo password enter karein. Sign in submit karein.

2. Organization workspace mein correct organization aur Owner role dekhein. Listings navigation par click karein. Direct URL /org/demo-organization-1/listings bhi use ho sakti hai.

3. Organization overview ke figures placeholder ho sakte hain. Orders/Analytics/Settings ko full implemented system na samjhein. Listings aur Team and access ke current actions test karein.

4. Sign out karein. Demo Buyer se same /login form par sign in karein. Buyer workspace → Discover materials aur Requirements dekhein.

5. Sign out karein. Demo Admin se sign in karein. Platform operations → Review queue open karein. Direct route /admin/review-queue. Initial queue empty normal hai: seeded stock already published hai.

Login	Expected workspace
demo-owner-1@resourcepk.example.invalid	Org demo-organization-1; create/edit/submit/archive
demo-buyer@resourcepk.example.invalid	Buyer; discovery + own requirements
demo-admin@resourcepk.example.invalid	Platform operations; pending listing reviews

Multiple roles ko safely compare karein

Simple method: ek role sign out, phir next role login. Same browser ki do tabs independent logins nahin hain; cookies shared hoti hain. Side by side test ke liye separate browser profiles use karein. Incognito windows bhi ek shared incognito session use kar sakti hain.

Workspace switcher current actual memberships aur platform assignments se views deta hai. Role dropdown edit karke admin access nahin milta. Owner/CEO, Manager aur Staff ke alag public signup buttons nahin hone chahiye.

Check: Owner own org kholta hai; Buyer platform review nahin kar sakta; platform reviewer queue khol sakta hai. Wrong password reject ho, stack trace na dikhe. Repeated login/verification/reset requests 429 rate limit de sakti hain; default 12 requests per 15 minutes hai.

13 Seller apna first product draft add kare

Demo Owner 1 login → Listings → Your materials. Yeh actual product add form hai. Neeche test data enter karein; yeh test record hai, real sellable inventory nahin.

Field label	Test input
Listing title	QA Cotton Lot 001
Material	Cotton
Condition	Clean production offcuts
Unit	kg
Available quantity	100.000
Minimum order quantity	10.000
Price per unit in PKR	210.29
Color optional	Natural
Composition optional	100% cotton
Location optional	Karachi
Description	Clean unused cotton offcuts for the Week 2 verification exercise.

1. Add price tier click karein. Tier 1 minimum quantity 10.000 aur PKR per unit 200.00 enter karein. Add price tier again: Tier 2 quantity 50.000 aur price 190.00.

2. Save draft click karein. Expected success message: Draft saved. Review it and submit for platform approval.

3. Inventory mein QA Cotton Lot 001, status draft aur correct values/tiers dekhein. Record refresh ke baad bhi rehna chahiye.

4. Signed out marketplace mein exact title search karein. Draft visible nahin hona chahiye.

5. Owner inventory → Edit draft. Description mein QA checked append karein → Save changes. Reload par edited value rehni chahiye.

Check: Quantity maximum 3 decimals aur price maximum 2 decimals; no silent rounding. MOQ 101 with stock 100 reject ho. Tier quantity stock se zyada, duplicate/decreasing threshold ya increasing price reject ho. Blank required description/title browser or API validation se stop ho.

Published stock ko direct edit nahin kar sakte; current editable states draft aur rejected hain. Eight images maximum; image controls Manage listing images draft/rejected card mein milte hain. Successful real upload section 21 mein check hoga.

14 Submit reject correct aur publish flow

1. Owner inventory par QA Cotton Lot 001 → Submit for review. Expected pending review. Edit/submit controls is state mein unavailable; public marketplace mein still hidden.

2. Sign out → Demo Admin login → Review queue. Listing correct organization, fields aur tiers ke saath visible honi chahiye.

3. Review reason for QA Cotton Lot 001 required field mein Correct description then resubmit likhein → Reject. Queue se item nikalna chahiye.

4. Sign out → Owner 1 login → Listings. Status rejected aur review note visible. Edit draft → correction save → Submit for review.

5. Admin login → Review queue. Required reason mein QA checked and approved enter karein → Publish. Queue item nikal jaye; record public searchable ho.

6. Signed out /marketplace par exact title search karein → View listing → reload. Correct material, quantity, MOQ, location, description aur tiers verify karein.

Transition	Expected boundary
Draft or rejected → submit	Owner/Manager own org only
Pending → publish/reject	Assigned platform reviewer only; reason min 3 chars
Published → edit	Current UI/API do not allow direct draft edit
Any non archived → archive	Owner/Manager confirmation; terminal state
Two editors stale version	409 conflict; reload and reapply intended change

Archive test last mein karein

Sections 15–16 matching complete karne ke baad Owner inventory → Archive this listing expand → Confirm archive. Archived record audit ke liye retained; republish nahin ho sakta. Public search, direct detail aur matching se remove ho. Detail request should be 404. Next repeat ke liye QA Cotton Lot 002 create karein.

Check: Public user draft/pending/rejected stock nahin dekh sakta. Owner apna item khud Publish nahin kar sakta unless separately authorized platform assignment ho. Account/organization status current database authorization se enforce hona chahiye.

15 Buyer discovery detail aur pagination verify kare

1. Sign out; /marketplace open karein. Search QA Cotton Lot 001 → Find materials. Published record visible ho.

2. Filters clear karein; Material Cotton, Unit kg select → Find materials. Search/filters change ke baad page 1 reset ho.

3. Sort Newest, Lowest price aur Highest price check karein. Order actual base listing price par hai; matching quantity ka effective tier price separate hai.

4. Initial seed 20 published records deta hai, default public page 12. No restrictive filter ke saath Next → page 2 aur Previous → page 1 test karein. Previous first page par disabled, Next last par disabled.

5. View listing click; URL save karein. Reload, new tab aur signed out session mein published details visible hon. Below MOQ/group buying wording informational hai; checkout currently available nahin.

6. Search title that does not exist. Clear no matches message hona chahiye, crash/fabricated fallback nahin. Filters reset karke results wapas laayein.

Protected collections ka pagination bhi check karein

Seller inventory, review queue aur buyer requirements default 20 per page hain; controls tab dikhte hain jab records limit se zyada hon. Demo Owner 1 ke 5 records par controls na dikhna normal hai. Section 17 se 21 test drafts import karke seller page 2 test kar sakte hain. Reviewer queue test ke liye 21 drafts submit karne honge; yeh optional larger manual run hai, isolated automated suite pagination contracts bhi check karti hai.

Buyer requirements page 2 check karne ke liye 21 disposable requirements create karein. Total/pages count stable ho; empty page, disabled control aur loading state sensible ho. Public/protected APIs page 1–200 aur limit 1–50 support karte hain; invalid bounds 422 hain.

Check: Search + material + unit + sort + paging + detail reload + empty results. Mark each separately. Listing images missing in initial demo normal; actual storage check Pass B mein hai.

16 Buyer requirements aur tier matching verify kare

Demo Buyer login → Requirements. Ensure QA Cotton Lot 001 section 14 se published hai aur archived nahin. Matching material comparison case insensitive exact material + same unit hai; free text synonyms/AI recommendations nahin.

1. Material Cotton, Quantity 50.000, Unit kg, Maximum PKR per unit 195.00 enter karein → Save requirement.

2. Requirement active visible ho. See matches click karein. QA Cotton Lot 001 include ho, matched price PKR 190.00 per kg, Meets current MOQ. Base 210.29 hone ke bawajood quantity tier price 190 applies.

3. Second requirement Cotton, quantity 50, kg, ceiling 189.00 save karein. QA lot is match list mein include na ho. Dusre seeded products match kar sakte hain; test specific QA title par hai.

4. Third requirement Cotton, quantity 5, kg, ceiling 220.00 save karein. QA lot match ho sakta hai with base 210.29; Below MOQ: group buying needed displayed. Is wording se purchase/pool actually create nahin hota.

5. Wrong unit meter ya quantity 101 ke requirement mein QA kg/100 lot exclude ho. Materials different ho to exclude ho.

6. Active requirement → Pause. Status paused aur See matches disabled. Resume → active; matches available. Close requirement expand → Confirm close. Closed terminal, no Resume; repeat ke liye new requirement create karein.

Check: Reload par requirements/status persist. Other buyer ke requirements visible nahin. Matching only published stock from active organizations, sufficient stock aur price ceiling ka applicable tier use karta hai. Matching maximum 50 results returns currently; result explanations transparent hain.

Requirement form optional ceiling omit kar sakta hai; notes API field hai lekin current form mein notes editor nahin. Requirements edit form delivered nahin; status operations aur new creation test karein.

Ab section 14 archive test karein. Archived QA lot requirement matches se remove ho. Old matches view manually See matches/reload se refresh karein; stale open view ko current server result na samjhein.

17 CSV aur XLSX stock import check karein

1. Owner 1 → Listings → Import a factory stock sheet → Download CSV template. Header order preserve karein. CSV UTF 8 mein save karein; Excel file ke liye Save As .xlsx, first sheet only.

2. Template mein neeche header aur valid row add karein. Commas wale text CSV mein double quotes ke andar likhein. Current template core eight columns only hai; images/tiers/optional metadata baad mein draft edit se add karein.

3. Factory stock sheet choose → Validate and preview. Expected 1 valid row, 0 errors. Is waqt database mein listing create nahin honi chahiye.

4. Confirm 1 draft listings click karein. One new draft visible; public discovery mein hidden. Edit draft, tiers/images add, submit/review separate actions hain.

5. Second file mein invalid row quantity 0 aur MOQ 200 with stock 100 add karein. Preview row/field errors dikhaye aur confirmation absent ho jab invalid rows hain. Correct file reupload karke preview repeat karein.

Destination: downloaded .csv file content, not terminal. Single data line neeche visual wrap ho sakti hai; save as one CSV line.

```
title,description,material,condition,unit,quantity,minOrderQuantity,pricePerUnit
QA Import Cotton 001,Clean cotton offcuts for local import
verification.,Cotton,Clean,kg,100.000,10.000,210.29
```

Boundary	Expected
Wrong header/order/extra column	Validation error; no records created
Wrong extension, corrupt workbook	Safe error; no crash or SQL writes
More than 100 data rows or 2 MB	Rejected; upload limit may be 413
Same confirmed request replay	No duplicate batch; API request key is idempotent
XLSX archive protection	Forged/corrupt/oversize zip cases covered by automated tests

CSV pagination fixture: use Excel fill series QA Pagination 01–21 with valid core values; import 21 rows once. Imported drafts still require review. Different repreview generates a new request key, so manually importing same file as a new action can legitimately create another batch.

18 Gmail SMTP aur real email checks prepare karein

Pass A demo does not need SMTP. Real signup, verification, forgot password aur Manager/Staff invitations ke liye actual mail delivery chahiye. Dedicated development Gmail account aur apne controlled recipient inboxes use karein.

1. Google account security mein 2 Step Verification enable karein. Official App Passwords page kholein. App name ReSource PK local test rakhein; generated app password privately copy karein.

2. App passwords option missing ho sakta hai for work/school policy, Advanced Protection ya restricted verification setup. Official Google guidance follow karein; normal account password substitute na karein.

3. Backend stop karein. backend/.env mein dono lines uncomment/fill karein. App password ki displayed grouping spaces remove karke actual value set karein.

4. backend/.env mein APP_URL aur CORS_ORIGINS localhost:3000 confirm; backend npm run dev restart karein. Frontend running reh sakta hai.

5. Apne seller, buyer, manager, staff recipient addresses prepare karein. Same address do roles public signup se banane ki bajaye invitation/account context use karein. Har email ke liye aap inbox access rakhte hon.

```
GMAIL_USER=YOUR_DEVELOPMENT_GMAIL_ADDRESS
GMAIL_APP_PASSWORD=YOUR_GOOGLE_APP_PASSWORD
```

Check: Both settings set hon, SMTP accepted aur actual inbox email receive ho. UI invitation sent message alone delivery proof nahin: delivery service pending/failure records bhi rakh sakti hai. Inbox aur email_deliveries section 22 se check karein.

Google App Passwords guidance

Google App Passwords page

User website password, Gmail normal password aur SMTP app password teen alag cheezen hain. Verification/reset/invite URL contains a bearer token; screenshot mein full URL na show karein. Token single use/expiry test section 23 mein hai.

Without SMTP signups may remain pending. No inbox email ho to verified state SQL se change karke PASS count mat karein; issue diagnose karein, SMTP fix karein, resend karein.

19 Real Seller Buyer signup verification aur reset

1. Sign out → /signup. Seller option select, Full name, Organization name, Email address aur strong Password fill → Create account. Screen delivery/verification instructions note karein.

2. Verification se pehle /login try karein: pending account access reject hona chahiye. Inbox/Spam check karein, app verification email link open karein. Verify result ke baad sign in karein.

3. Seller account organization workspace mein Owner hona chahiye. Org slug email link token nahin; actual workspace URL note karein. One QA draft create/save karke own org persistence check karein.

4. Sign out → /signup → Buyer select. Separate controlled email se create account → inbox verification → /login. Buyer workspace and own requirements available hone chahiye.

5. Forgot password link /forgot-password par real Buyer email enter karein. Inbox reset link open karein → new strong password set. New password works; old password reject. Another browser/profile ki prior session next authenticated request par revoked honi chahiye.

6. Used verification/reset link reopen karein: reject or safe already processed state; duplicate action nahin. Resend verification /resend-verification pending test account ke liye use karein. Unknown email reset response account existence disclose na kare.

Buyer se seller banne ka current known issue

Buyer overview → Start selling with this account → Organization name → Create seller organization flow exists. Source review found backend createOrganization response returns id/name but not slug, while frontend immediately navigates to organization.slug. Is snapshot par success ke baad /org/undefined/listings redirect ho sakta hai. Yeh finding code inspection se hai; is guide session mein live reproduction nahin ki gayi.

Recovery: organization creation dobara repeat na karein. Sign out/sign in ya /workspace reload se refreshed membership ka real org route select karein. Correct fix backend response mein slug include karna aur onboarding redirect regression test add karna hai; is finding ko FAIL/known issue record karein. Main seller signup path is separate.

Check: Real signup + pending denial + inbox verification + login + reset + old password/session rejection. Demo emails in tests use na karein: .invalid domain delivery nahin karta.

20 Manager Staff aur CEO portal ka full check

CEO current Owner role hai; separate CEO signup/URL nahin. Manager/Staff public signup se self assign nahin hote. Real organization Owner login karein aur sidebar Team and access open karein; route /org/<your slug>/team-access.

1. Work email mein controlled Manager email enter, Role manager choose, Allow this manager to invite staff checkbox check → Send secure invitation.

2. Inbox invitation open karein. Owner currently signed in ho to Sign out to use the invited account click karein. New invitee Full name + password enter karke join kare; existing invitee matching verified account se sign in → Accept as invited account.

3. Manager login/context: same organization, role Manager. Listings par QA Manager Draft create/edit/save/submit test karein. Platform Publish access unavailable hona chahiye.

4. Manager → Team and access. Staff option only. Controlled Staff email enter → invitation send → invited account accept. Manager ko Manager/Owner/platform role invite karne ki permission nahin.

5. Staff login: correct organization shell. Team access invite action denied; Listings seller actions/API currently denied. Assigned stock/orders future scope hai, is snapshot mein Staff inventory edit claim nahin.

6. Owner ek second Manager invite kare without checkbox. That Manager listing operations kar sakta hai, Staff invitation nahin. Owner Staff ko directly bhi invite kar sakta hai.

Negative test	Expected
Different email signed in on invite	Mismatch shown; acceptance blocked
Used or expired invitation	Rejected; membership duplicate nahin
Invitation reissued same email	Old invitation revoked; latest only
Manager without delegation	Staff invite denied
Staff or other organization user	Cannot mutate this org stock/team
Owner role transfer/full member CRUD	Not delivered in current invitation UI

Invitations 72 hours aur single use hain. Expiry ke liye real time wait ya isolated automated test use karein; production database timestamps/roles manually edit na karein. Owner/Manager/Staff role ka visible label alone proof nahin, server authorization check bhi required.

21 Private Supabase images configure aur verify karein

Local business database reh sakta hai; images separate authorized Supabase project ke private test bucket mein store honge. Yeh live provider action hai. Production/shared inventory par destructive experiments na karein. Supabase skill/provider documentation private bucket aur backend key boundary govern karti hai.

1. Supabase dashboard → authorized project select. Project URL aur service_role key backend credential page se privately obtain karein. Public anon/publishable key service role substitute nahin. Settings ke exact screen names change ho sakte hain; official links section 30 par hain.

2. Backend stop. backend/.env mein all three settings fill karein. Disposable bucket name listing-images-preview recommended; existing project bucket listing-images use karne se pehle scope verify karein.

3. Backend terminal npm run storage:setup chalayein. Script bucket create/update karta hai: private, WebP only, max 5 MB. Supabase Storage dashboard mein actual restrictions check karein. Then npm run dev restart.

4. Owner draft → Manage listing images → Listing image choose small JPEG/PNG/WebP, Image description Cotton offcuts on table → Upload image. Card/reload par rendered image and alt description verify.

5. Draft signed out public listing/image access blocked ho. Reviewer pending item ke authorized view mein image visible ho. Publish ke baad public listing signed image visible ho.

6. Separate disposable draft par Remove image test karein. Database link removed; provider object deletion immediate or durable cleanup se complete ho. Eligible retry jobs two minutes baad npm run storage:cleanup; provider object absence and queue result check karein.

```
SUPABASE_URL=https://YOUR_PROJECT_REF.supabase.co
SUPABASE_SERVICE_ROLE_KEY=YOUR_BACKEND_ONLY_SERVICE_ROLE_KEY
LISTING_IMAGE_BUCKET=listing-images-preview
```

Check: Max 8 images; input 5 MB, 16 MP, JPEG/PNG/WebP only; actual decode and WebP conversion (max dimensions 1600). Animated/invalid raster/SVG rejected. Signed URLs 60 seconds, no store response; reload/Retry image fresh link fetch kare. URL expiry uploaded object delete nahin karti.

No antivirus provider claim kiya gaya. Provider timeout recovery simulated automated evidence hai; real service upload/signed read/delete/cleanup separately PASS chahiye. Without credentials upload error expected, successful storage verification BLOCKED.

22 Database ko read only queries se inspect karein

Destination: pgAdmin → resourcepk_preview → Query Tool. Queries below read only hain. app schema tables public schema/Supabase Auth users se different hain. Tokens, password_hash ya refresh hashes select/export na karein.

```
SELECT name, applied_at FROM app.schema_migrations ORDER BY name;
SELECT kind, count(*) FROM app.demo_seed_records GROUP BY kind;
SELECT status, count(*) FROM app.listings GROUP BY status;
SELECT email, status, email_verified_at FROM app.users ORDER BY email;
SELECT o.slug, m.role, m.status, m.can_invite_staff
FROM app.organization_memberships m
JOIN app.organizations o ON o.id=m.organization_id
ORDER BY o.slug,m.role;
```

Check: Five migration names 001–005. Demo markers users 6, organizations 4, listings 20 stay stable. Manual new records totals increase; demo marker counts do not. Real verified accounts active + verified timestamp; invited memberships correct org/role.

Exact values constraints aur audit

```
SELECT title,status,quantity,min_order_quantity,price_per_unit,
       price_per_unit_minor,version,is_demo
FROM app.listings WHERE title LIKE 'QA %' ORDER BY created_at;
SELECT count(*) AS invalid_moq FROM app.listings
WHERE min_order_quantity > quantity;
SELECT count(*) AS orphan_images FROM app.listing_images i
LEFT JOIN app.listings l ON l.id=i.listing_id WHERE l.id IS NULL;
SELECT action,resource_type,created_at FROM app.audit_events
ORDER BY created_at DESC LIMIT 30;
SELECT status,count(*) FROM app.email_deliveries GROUP BY status;
SELECT count(*) AS pending_cleanup FROM app.image_cleanup_jobs;
```

Check: QA price 210.29 stores price_per_unit_minor 21029; version increments after mutation. Invalid MOQ/orphan image counts zero. Listing/import/review/requirement actions audit recorded. Email sent state plus actual inbox delivery; failed/pending must investigate.

Migrations applied files: 001_phase1_foundation.sql, 002_advisor_hardening.sql, 003_catalog.sql, 004_catalog_completion.sql, 005_catalog_foreign_key_index.sql. Migration failure par manually half schema patch na karein. Backup/restore rehearsal full release work remains; fresh disposable database rebuild separate from real data recovery hai.

23 Authentication session aur permission security checks

Test	Baby steps aur expected result
Password validation	Real signup weak password, missing upper/lower/number or too short; browser/API reject. Max 72 UTF 8 bytes, not simply characters.
Pending login	Unverified real account login denied; inbox verification required.
Session reload	Login → F5 → protected workspace still correct. Access token memory; refresh cookie HttpOnly.
Logout	Logout → protected URL → login redirect; protected API 401. Old refresh session cannot continue.
Cookie flags	DevTools Application → Cookies. Local HttpOnly, SameSite Lax; production HTTPS Secure + SameSite None. Never copy cookie value.
Refresh CSRF	Refresh/logout origin and required custom CSRF header enforced. Missing/wrong values reject; automated tests cover.
Buyer vs platform	Buyer /admin route or review endpoint cannot access moderation. Owner not automatically platform reviewer.
Other organization	Owner 2 using Owner 1 slug/listing/org UUID cannot view/mutate private inventory. API should deny, no data leak.
Staff restrictions	Staff inventory mutation/invitations denied even if request manually crafted.
Stale version	Open own draft in two tabs; save first then second old edit. 409; no lost overwrite. Reload before retry.

Identity checks requiring isolated fixtures

Refresh replay, expiry, suspended account/org/membership, invitation inviter grant revoked before acceptance aur password reset revoking sessions automated integration fixtures se verify karein. In tests ke liye staging roles or statuses manually edit na karein. Owner UI complete suspension/member editor abhi nahin hai.

Normal local demo all accounts ka password same chosen test password hai, isliye production identity design ka sample nahin. Live accounts individually private strong passwords use karein. Seed production mein disabled hona chahiye.

Check: Default auth limiter 12 requests/15 min aur public reads 300/min. Excess requests 429 protection ho sakti hai. Limiter in process hai; distributed multi instance limiter, high load SLA aur exhaustive penetration audit delivered nahin. Automated pass absolute security guarantee nahin.

24 API failure aur leak checks

Destination: new Git Bash terminal while manual servers running. Anonymous checks safe local endpoints par hain. curl Windows Git Bash mein available hota hai. Status headers -i se print honge.

Public API and invalid pagination

```
curl -i 'http://localhost:5000/api/v1/listings?page=1&limit=12'
curl -i 'http://localhost:5000/api/v1/listings?page=0'
curl -i 'http://localhost:5000/api/v1/listings?limit=51'
```

Check: First 200, data wrapper with items/total/page/limit. Invalid bounds 422 VALIDATION_ERROR and correlationId, no stack/SQL/password values.

Anonymous protected data and wrong origin

```
curl -i http://localhost:5000/api/v1/requirements
curl -i -H 'Origin: https://invalid.example' \
  http://localhost:5000/api/v1/listings
curl -i http://localhost:5000/api/v1/internal/storage-cleanup
```

Check: Protected requirements 401. Unapproved Origin 403 ORIGIN_NOT_ALLOWED. Internal cleanup no secret 401. CORS is not authorization: no Origin public listing remains intentionally public, protected routes still require session/JWT.

Input and upload checks

Own local draft par negative price, price 210.299, quantity 0, MOQ above stock, bad unit, unexpected fields aur malformed UUID should safely reject. UI block ko API enforcement proof na samjhein; regression/integration tests server validations cover karte hain. Larger-than-limit raster/CSV rejects without new DB rows or orphan files.

Short valid HTML looking text description/title inert text ki tarah render hona chahiye, code execute nahin. Real malicious payload production par use na karein. CSV/XLSX corruption/zip limits isolated suite se check karein. Parameterized SQL aur DB constraints source review ka hissa hain.

Check: Generic server error contains request ID and safe message. No password, access/refresh tokens, service role key, DB URL or stack trace browser response mein. Frontend built JS mein backend secrets present nahin hone chahiye. Do not upload .env to GitHub.

25 Responsive keyboard aur browser behavior check karein

1. Chrome/Edge F12 → device toolbar Ctrl+Shift+M. Responsive mode → width 360, phir 390, 768, 1024, 1280, 1440 pixels test karein. Height 900 convenient; physical device test separate hai.

2. Har width par home, signup/login, marketplace, detail, seller form/inventory, review queue aur buyer requirements open karein. Horizontal overflow, cut text, overlapping controls aur unusable tables check karein.

3. Narrow workspace par menu open/close. Tab/Shift+Tab focus stays inside open drawer, Escape closes, focus opener par return; menu links correct route kholein.

4. Keyboard only login/form/price tiers/import/review complete karein. Labels visible, focus outline, disabled controls sensible aur validation error readable ho.

5. DevTools Network → Offline briefly, search/collection load karein. Visible error + Try again expected. Online restore → retry succeeds. Images failing hon to Retry image/reload. Do not count blank screen as acceptable failure.

6. Reduced motion preference emulate karein. Static motion fallback usable ho; 3D/decorative layers critical forms or content ko block na karein.

Screen group	360 390 768 1024 1280 1440 results
Public + detail	_____ _____ _____ _____ _____ _____
Login + signup	_____ _____ _____ _____ _____ _____
Seller + tiers + images	_____ _____ _____ _____ _____ _____
Reviewer	_____ _____ _____ _____ _____ _____
Buyer requirements	_____ _____ _____ _____ _____ _____

Eight Chromium scenarios already six widths par passed evidence hai. That does not establish full Safari/Firefox, physical mobile devices, screen reader, Urdu, 200% zoom, slow network aur full accessibility audit. Aap Edge/Chrome ke baad available Firefox/Safari/physical mobile par same main flow record karein; not run ko PASS mat likhein.

Check: At 200% browser zoom form/actions still reachable. Errors/loading/empty states visible. Actual provider image timing and real mail handling hosted/device pass mein separate record karein.

26 Automated tests ko isolated databases par run karein

First manual backend/frontend terminals Ctrl+C se stop karein. PostgreSQL stays running. pgAdmin mein section 7 jaisi creation se resourcepk_test_integration aur resourcepk_test_e2e databases create karein, Owner postgres. Tests disposable records reset kar sakte hain; resourcepk_preview ya remote DB mat use karein.

Tests se pehle backend/.env mein GMAIL_USER, GMAIL_APP_PASSWORD, SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY aur LISTING_IMAGE_BUCKET lines ke start par # laga kar temporarily comment karein, save karein. Empty values mat set karein. Real secrets isi ignored file mein rahein. E2E expects STORAGE_NOT_CONFIGURED; real provider settings se yeh test fail hoga.

Backend quick checks

```
cd /c/Projects/ReSourcePK-preview/backend
npm run lint
npm run format:check
npm test
npm audit
```

Real PostgreSQL integration suite

Below URL mein locally encoded password replace karein. New testing terminal use karein; env export .env defaults override karta hai. Local test hostname + resourcepk_test_ name enforced hai.

```
export NODE_ENV=test
export TEST_DATABASE_URL='postgresql://postgres:YOUR_ENCODED_DB_PASSWORD@127.0.0.1:5432/
resourcepk_test_integration'
export DATABASE_URL="$TEST_DATABASE_URL"
node scripts/prepare-test-db.js
npm run test:integration
```

Frontend checks and production build

Destination: Frontend terminal, frontend/.env still local API URL.

```
cd /c/Projects/ReSourcePK-preview/frontend
npm run lint
npm run format:check
npm run build
npm audit
```

Check: Snapshot evidence: backend 27 regression tests + 12 PG scenarios; frontend build/lint/format and full audits passed. Zero known audit vulnerabilities then does not promise future audit stays zero; review new result. Frontend no standalone npm test/typecheck script currently.

Integration provider transport may be simulated; DB transactions and image decoding actual. Successful tests do not prove Gmail/Supabase/Vercel live credentials. On failure keep first error and fixture DB name; do not point tests at staging to make them work.

27 Chromium full journey aur hosted sign off

Destination: Backend testing terminal. Both manual servers must be stopped. Frontend must already npm run build from section 26. Six demo users are created only inside isolated E2E DB.

```
cd /c/Projects/ReSourcePK-preview/backend
export NODE_ENV=test
export TEST_DATABASE_URL='postgresql://postgres:YOUR_ENCODED_DB_PASSWORD@127.0.0.1:5432/
resourcepk_test_e2e'
export DATABASE_URL="$TEST_DATABASE_URL"
export ALLOW_DEMO_SEED=true
read -s -p "Choose strong E2E password: " E2E_PASSWORD
echo
export E2E_PASSWORD
export DEMO_PASSWORD="$E2E_PASSWORD"
export AUTH_REQUESTS_PER_WINDOW=100
node scripts/prepare-test-db.js
npm run db:seed:demo
node node_modules/@playwright/test/cli.js install chromium
npm run test:e2e
node node_modules/@playwright/test/cli.js show-report
```

Expected 8 scenarios pass, main journey all six widths. Report stop Ctrl+C. AUTH_REQUESTS_PER_WINDOW=100 isolated tests only; normal default increase na karein. Close testing terminal, backend/.env mein section 26 ke five provider lines ka temporary # remove karke save karein. New terminals normal preview DB par start karein. Windows Chromium install mein --with-deps nahin chahiye.

Hosted acceptance abhi separately required

Authorized Vercel project access restore karein. Frontend VITE_API_URL hosted API/api/v1, backend APP_URL/CORS exact hosted frontend, strong JWT, DB pooler SSL, SMTP/storage all configured; backend provider secrets VITE mein nahin. No demo seed in production. Migrate once reviewed; deploy paired contracts and users sign in again.

CRON_SECRET independent random backend secret 32–256 chars set karein. Versioned vercel.json daily 0 3 * * * UTC cron config uses protected GET internal/storage-cleanup. Vercel cron production only; Hobby execution may occur within scheduled hour. Observe actual authorized invocation/logs and provider object cleanup. CLI storage:cleanup is manual maintenance, scheduler proof nahin.

Check: Hosted seller → image → submit → reviewer publish → public detail/search → buyer tiers/requirements → pause/close/archive; real email/reset/invite; production cookie/CORS; storage private/signed/delete; scheduled job log. Any access/credential missing means BLOCKED. Preview/local CI cannot substitute this sign off.

28 Common errors aur exact recovery

Problem	Meaning aur next step
git/node/npm not found	Install correct official tool; close/reopen VS Code/terminal; node --version check.
npm engine error	Use Node 24. Correct repo directory aur npm ci --ignore-scripts. Lockfile randomly rewrite na karein.
Sharp missing binary	Supported Node/Windows architecture verify. Optional native packages must install; npm config omit should not omit optional. Re run npm ci --ignore-scripts; do not globally enable install scripts.
Database password auth failed	Postgres user/password/port check; encoded password URL mein; wrong local vs pooler user mat mix karein.
Database does not exist	pgAdmin create correct exact resourcepk_preview name; DATABASE_URL pathname verify.
role anon/authenticated missing	Local section 7 NOLOGIN SQL run, then migrate again. Hosted Supabase roles manually recreate nahin.
relation/column missing	Five migrations and correct connected DB verify; do not use legacy schema.sql.
Health ok, ready fails	API running, database not connected. PostgreSQL Windows service running + URL/credentials check.
Port already in use	Old server terminal Ctrl+C. Manual servers stopped before Playwright. Use ports 3000/5000 consistently.
CORS or fetch failed	API running, VITE_API_URL /api/v1 ending; localhost vs 127 origin mismatch; CORS exact frontend URL. Restart after .env edit.
Pending/unverified login	Configure SMTP + actual inbox verify. Demo chosen password exists only seeded demo accounts.
Password changed by seed?	Replay preserves old password. Use first seed password; real account reset requires inbox.
Invitation sent but no mail	Inbox/Spam and email_deliveries failed/pending inspect; fix GMAIL settings. UI message alone insufficient.
429 after repeated login	Rate limiter, normally 15 min window. Wait then retry; test suite isolated budget only.
409 conflict	Stale version or terminal status; reload latest record, reapply allowed action.
Upload 413/422/503	Size/format/validation or missing/unavailable storage. Small real raster + all three settings + bucket check.
Start selling redirects undefined	Known response slug omission, section 19. Refresh workspace; do not recreate org.
Vercel 403	Team membership/plugin/CLI scope needs authorized access; report BLOCKED, not deployment success.

Record first failing command/status, sanitized error and request ID. Avoid reinstalling everything or deleting databases before root cause is clear. Share no .env, full cookies/tokens or raw HAR.

29 Daily restart final checklist aur bug report

Agli baar project start

PostgreSQL Windows service running confirm. Fresh Git Bash terminals open so testing exports are gone. No need daily seed/migration on unchanged checkout; current migration command safe skip karta hai. Before updates read migration/paired API contract notes.

```
cd /c/Projects/ReSourcePK-preview/backend
npm run dev

cd /c/Projects/ReSourcePK-preview/frontend
npm start
```

Browser localhost:3000, API /ready check. Stop each server Ctrl+C. Do not close database/delete project just to stop app. Repo update chahiye to git status clean confirm, feature branch verify, git pull --ff-only; dependencies change par npm ci --ignore-scripts, new migrations par db:migrate, frontend build retest.

Final gate	PASS FAIL BLOCKED NOT RUN
Correct branch, dependencies, .env private	_____
Five migrations + seed replay + /ready	_____
Owner create/edit/submit/reject/resubmit/publish	_____
Public discovery/detail/pagination/archive	_____
Buyer tier matches/pause/resume/close/isolation	_____
CSV + XLSX preview/error/confirm/replay	_____
Real signup/verify/reset + Manager/Staff invites	_____
Real private images/read/remove/cleanup	_____
Auth/tenant/failure/responsive/keyboard checks	_____
27 unit + 12 integration + 8 browser checks	_____
Hosted auth/storage/cron + access settings	_____

Bug report template

Title _____ Date/time/timezone _____ OS/browser _____ Backend/frontend SHA _____ Local/hosted _____ Role/org _____ Step number _____ URL without tokens _____ Expected _____ Actual _____ HTTP/error code _____ Request ID _____ Sanitized screenshot/log _____ Reproducible yes/no _____

30 Source map evidence aur official references

Current project checklist is 11/37 checked, not all roadmap done. Week 1/2 code completion aur hosted release sign off alag evidence hain. Known buyer onboarding redirect fix and remaining hosted gates resolve hone ke baad docs/tasks/status update aur reviewed paired merge required hai.

Need to inspect	Repository file or document
Requirements and scope	Both repos docs/README.md, prd.md, srs.md, tasks.md, memory.md
Stack UI rules and reporting	docs/architecture.md, design.md, rules.md, analytics.md
Catalog API validation/data	Backend src/routes/catalogRoutes.js, src/modules/catalog/validation.js, service.js, repository.js
Auth and org boundaries	Backend src/routes/authRoutes.js, src/services/authService.js, organizationService.js, src/middlewares/authMiddleware.js
Schema and fixtures	Backend migrations/001–005; scripts/seed-demo.js, demo-fixtures.js, verify-demo.js
Storage and scheduler	Backend src/modules/catalog/imageAdapter.js, imageCleanupService.js, src/routes/maintenanceRoutes.js, scripts/setup-storage.js, cleanup-images.js, vercel.json
User screens	Frontend src/components/Catalog.jsx, CatalogMedia.jsx; src/pages/ListingDetail.jsx, Workspace.jsx and auth pages
Acceptance and limits	Both docs/verification-week-2.md, delivery-status.md, adr/004-catalog-completion.md, adr/005-week-2-acceptance.md
Reproducible tests	Backend test/, integration/, e2e/, playwright.config.js; both .github/workflows/

Backend PR

Frontend PR

Backend latest recorded CI

Frontend latest recorded CI

Supabase database connections

Supabase private storage

Vercel cron security and logs

This guide was built from the current branch source and README linked acceptance docs. The command sequence was checked against package scripts/configuration; fresh Windows install and real hosted credentials were not executed in this guide session. Existing CI links are prior automated evidence, not a claim of new live provider tests.

31 Optional Supabase database se local app run karein

Recommended first run section 7 ka local database hai. Agar aap Supabase database use karna chahte hain to yeh alternative follow karein. Automated integration/E2E tests phir bhi dedicated local PostgreSQL databases par hi chalenge. Database source switch karne se accounts/stock bhi switch hote hain; local seed remote DB mein automatically nahin aata.

1. Supabase dashboard par authorized development project open karein. Prefer disposable dev project; existing ReSourcePK Staging shared project par real users/inventory change karne se pehle intended test scope verify karein. Production par seed/tests mat chalayein.

2. Project ke Connect dialog mein PostgreSQL connection string dekhein. Backend README baseline shared transaction pooler URL use karta hai, normally port 6543. Actual dashboard host aur username copy karein; postgres.<project ref> ko local postgres username se replace na karein. Direct connection may need IPv6; network limitations mein pooler use karein.

3. Project database password use karein, Supabase login/GitHub/website password nahin. Password lost ho to authorized project database password reset action use karein; existing integrations update honge, isliye shared project par impact check karein. Section 7 hidden prompt se URL encoding karein.

4. Both app servers stop karein. backend/.env DATABASE_URL ko actual complete pooler string se replace karein; SSL required connection use karein. Local APP_URL/CORS still http://localhost:3000, NODE_ENV development aur pool max 5 rakhein. Frontend VITE_API_URL localhost:5000/api/v1 hi rahega.

5. Backend terminal npm run db:migrate run karein. Existing staging 001–005 already applied hon to successful no op. Supabase anon/authenticated roles already provisioned hote hain; local role creation SQL yahan run na karein.

6. Backend npm run dev aur frontend npm start. /ready status ready check. Supabase SQL Editor mein section 22 read only app schema queries run karein. Private app schema ko Data API mein expose ya public grant karke UI fix na karein; Express API database access own karta hai.

7. Real accounts section 18–20 SMTP verification/invitations se create karein. Existing trusted platform reviewer authorized admin se provision hota hai; public signup se self grant platform access nahin. Local demo admin remote DB par exist nahin jab tak seed explicitly allowed dev fixture mein run na ki gayi ho.

8. Is optional path mein DB not reachable par Connect URL, encoded password, project active state, SSL aur pooler username/port check karein. Authentication tables Supabase Auth dashboard mein nahin, private app.users mein hain. Local preview par wapas aane ke liye backend .env old local URL restore aur server restart karein.

Check: Supabase /ready success, five migrations, API served records correct project se, keys browser mein absent. Live reviewer missing ho to moderation BLOCKED mark karein; permissions bypass karke PASS nahin. Providers ya team access unavailable hone par local demo pass still useful hai, hosted release sign off pending rahega.